

Paytm Lakehouse Platform — Working Design

1. Requirements, Scope & Consumers

The goal is to build a unified lakehouse platform capable of ingesting and serving data across Paytm-style business domains while keeping the data trustworthy, current, auditable and cost-efficient.

Source Families

The platform supports five ingestion families:

- Transactional databases
- Event streams
- Partner/vendor file drops
- Third-party APIs
- Operational spreadsheets

Application telemetry and business events can use the event-stream ingestion pattern where appropriate.

Consumer Personas

Analysts / Finance

- dashboards and KPIs;
- historical analysis;
- ad-hoc SQL;
- financial reconciliation;
- governed business metrics.

Applications

- consume trusted data through Gold/warehouse-backed services or a specialized serving store;
- seconds-level response is sufficient for the assumed application workloads;
- no hard millisecond/microsecond key-value lookup requirement is assumed.

Data Scientists

- offline feature engineering;
- historical analysis;
- model training.

Auditors

- lineage and provenance;
- transaction traceability;
- DQ and reconciliation evidence;
- pipeline execution history;
- access/security history;
- end-to-end auditability.

2. Assumptions & Back-of-the-Envelope Sizing

2.1 Working Assumptions

- Standard analytical reporting requires **minutes-level freshness**.

- Finance prioritizes correctness over latency.
- Some operational dashboards over event data may require **few-second freshness**, for example recent order/payment aggregates.
- Application response latency can be in seconds; hard millisecond or microsecond lookup is not assumed.
- Data Scientists primarily consume offline Silver/Gold datasets; no online feature store is assumed.
- Transactional CDC requires minutes-level lakehouse freshness.
- CDC volume is assumed to be approximately **100M changes/day** for sizing only.
- Average event size is assumed to be approximately **1 KB**.
- Average external-file row size is assumed to be approximately **500 bytes**.
- Average event-stream load is assumed to be approximately **3K events/sec**, against the brief-provided ~10K/sec peak.
- API and spreadsheet volume is assumed to be relatively small compared with the 500M file-row/day envelope.
- Approximately **2.5–3× raw/source-equivalent storage** is assumed after Raw, transformed lakehouse layers and selected serving copies.
- Five years of history is retained. Approximately one year is assumed to remain in hot/actively queried storage; older data can move to cheaper cold/archive storage.
- A broad **minute-level recovery objective** is assumed rather than committing to an exact RTO/RPO.
- Standard enterprise PII controls are assumed. No additional residency/deletion requirement beyond the brief is assumed.

2.2 Capacity Inputs

Item	Provided / Assumed	Value
Event peak	Brief	~10K events/sec
Event average	Assumption	~3K events/sec
Event size	Assumption	~1 KB
File volume	Brief	~500M rows/day
File row size	Assumption	~500 bytes
Customers	Brief	~50M
CDC volume	Assumption	~100M changes/day
CDC record size	Assumption	~1 KB
Retention	Brief	5 years
Money representation	Brief	Integer paise

2.3 Derived Sizing

At the above working assumptions:

- event data ≈ **259M events/day**;
 - combined event + file + CDC volume ≈ **859M records/day**;
 - raw/source-equivalent data ≈ **0.6 TB/day**;
 - five-year raw/source-equivalent footprint ≈ **1.1 PB**;
 - approximate total lakehouse/serving footprint at 2.5–3× ≈ **2.8–3.3 PB**.
- These are directional sizing numbers, not infrastructure commitments.

3. Architecture Overview

The architecture is designed as **one unified platform with multiple source-specific ingestion adapters**.

The source mechanisms are different, but after landing they converge into common storage, processing, governance, and serving patterns.

Event streams additionally support a real-time path:

Paytm Lakehouse Platform

ingestion, processing, lakehouse and serving architecture

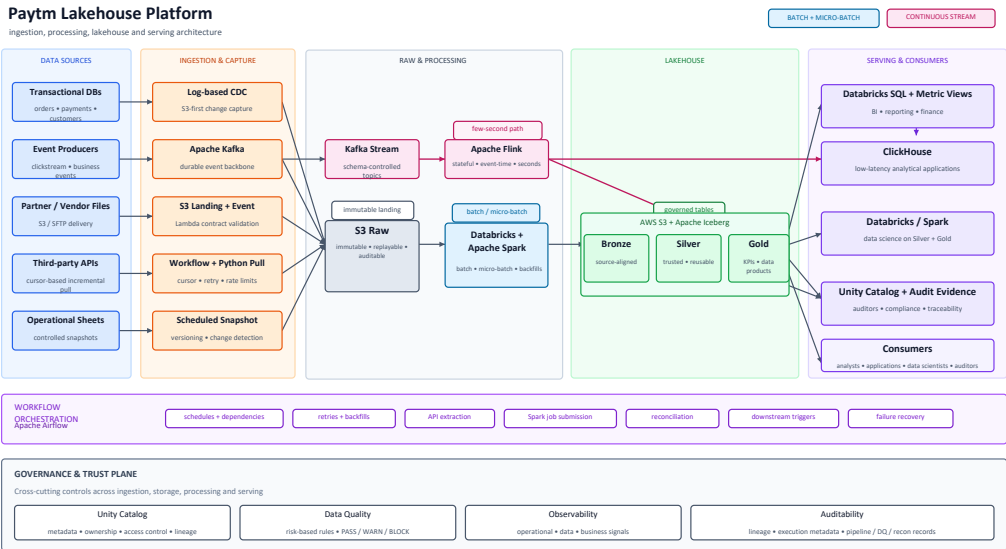


Figure 1. Paytm Lakehouse Platform - ingestion, processing, lakehouse and serving architecture

Kafka → Flink → ClickHouse

for continuously updated operational aggregates where the normal lakehouse path is not fresh enough.

4. Unified Ingestion Framework

We use **one ingestion framework with source-specific adapters**. Each source tracks arrival and progress differently, while sharing common controls for validation, idempotency, audit, reconciliation and recovery.

4.1 Common Ingestion Principles

- Immutable landing:** We preserve source data in immutable object storage such as **Amazon S3** for replay, audit and reprocessing.
- Lakehouse conversion:** After validation, we organize data into **Apache Iceberg** tables using a Bronze–Silver–Gold Medallion architecture.
- Progress tracking:** We track source-native progress through CDC checkpoints, Kafka offsets, object versions, API cursors or snapshot versions.
- Controls & recovery:** All adapters validate contracts, maintain idempotency, capture audit evidence, reconcile expected vs processed data, quarantine critical failures and support replay.
- Orchestration:** We use **Airflow** where schedules, dependencies, retries, reconciliation or backfills require workflow coordination.

4.2 Transactional Database Adapter

Primary Flow

Transactional DB → Log-based CDC → Immutable S3 Raw → Spark Micro-batch → Iceberg Bronze → Silver/Gold

A CDC service such as **AWS DMS** reads transaction logs—e.g. PostgreSQL WAL, MySQL binlog or Oracle redo/SCN—and lands incremental changes without repeatedly scanning source tables.

Trigger & Progress State

The CDC service maintains the source log checkpoint, such as LSN/binlog position/SCN, and resumes from the last committed position. **Airflow → Spark** processes newly landed change ranges as micro-batches.

Controls & Recovery

We validate schema, business keys and CDC operations; capture checkpoint/range, row counts and run IDs for audit; and reconcile landed changes against applied records. Spark uses idempotent **MERGE**/upsert semantics, so retries can safely replay the same change range. Transformation defects are corrected by replaying the affected immutable CDC history.

Alternative / Trade-off

We choose **S3-first CDC** because minutes-level freshness does not justify another always-on streaming path.

For tighter latency, burst buffering or multiple independent consumers:
DB → CDC Connector → Kafka → Spark Structured Streaming → Iceberg

Kafka improves decoupling and latency but adds continuous infrastructure and operational cost.

4.3 Event Stream Adapter

Primary Flow

Events enter a durable streaming backbone such as **Kafka** and follow two paths:
Historical: Kafka → Immutable S3 Raw → Spark → Iceberg Bronze → Silver/Gold
Real-time: Kafka → Flink → Real-time Serving

We retain the S3 path for durable history, replay and audit, while Flink handles stateful/event-time workloads requiring few-second freshness.

Trigger & Progress State

Kafka tracks progress through **topic, partition and offset**; Flink checkpoints persist source positions together with operator/window state.

Controls & Recovery

We enforce event contracts through a **Schema Registry** and retain **event_id**, topic, partition, offset and schema version for traceability. Stable event IDs, event time and watermarks handle duplicate and late events. Selected real-time outputs can be reconciled against the historical lakehouse path. Flink restores from its latest successful checkpoint; long-term replay and historical correction use the immutable S3 history rather than Kafka retention.

Alternative / Trade-off

We use **Flink** where state, windows and late/out-of-order processing are first-class requirements.
Spark Structured Streaming remains the simpler alternative when continuous lakehouse ingestion and technology consolidation matter more.

4.4 Partner / Vendor File Adapter

Primary Flow

Partner → Object Storage → Arrival Event → Lightweight Validation → Spark → Iceberg Bronze → Silver/Gold

On AWS:
Partner → S3 → S3 Event/SQS → Lambda → Airflow/Spark
Lambda performs lightweight pre-flight checks; Spark performs large-scale processing.

Trigger & Progress State

We identify each delivery through **source + object path + version/ETag/checksum** and maintain its arrival and processing state in an ingestion-control table. This is separate from Iceberg manifests, which describe files committed inside Iceberg tables.

Controls & Recovery

We validate filename/type, schema, mandatory columns, readability and object identity, then reconcile:
expected/landed → registered → processed

This detects missing files, duplicate deliveries, missed triggers and stranded processing while maintaining audit evidence.
Critical failures are quarantined; retries reuse the immutable object. Partner corrections arrive as **new source versions** rather than overwriting history.

Alternative / Trade-off

We use event-driven triggering for responsiveness and scheduled reconciliation as the completeness safety net.

4.5 Third-Party API Adapter

Primary Flow

Workflow → API Extraction → Immutable S3 Raw → Spark → Iceberg Bronze → Silver/Gold

An **Airflow/Python** task handles external API communication; Spark workers process the landed data rather than independently calling the vendor.

Trigger & Progress State

We use source-supported incremental state such as **cursor, continuation token or updated-since timestamp**, advancing it only after the corresponding payload is safely landed.

Controls & Recovery

The ingestion task handles authentication, pagination, rate limits, **Retry-After**, bounded retries and response/schema validation.
We persist request range/cursor, response identity, record count, landing status and committed cursor for audit and reconcile extraction ranges where required.
A failure before landing does not advance the cursor. Downstream failures replay Raw; vendor corrections are re-fetched and landed as new immutable versions.

Alternative / Trade-off

We default to scheduled pull and use **webhooks/event delivery** where the provider offers a reliable push mechanism and lower latency is required.

4.6 Operational Spreadsheet Adapter

Primary Flow

Scheduled Extraction → Immutable Snapshot → Validation → Spark → Iceberg Bronze → Silver/Gold

We treat sources such as **Google Sheets, CSV and Excel files** as versioned snapshots rather than transactional CDC streams.

Trigger & Progress State

Airflow extracts each source on schedule and identifies the snapshot using source version/timestamp plus a content hash.

Controls & Recovery

We apply stronger schema and business-rule validation because these sources are human-maintained, including mandatory fields, datatypes, duplicate keys and structural changes.
Snapshot version, row/change counts, validation result and processing state provide audit and reconciliation evidence.
Invalid snapshots are quarantined; retries reuse the same immutable snapshot, while corrections arrive as new versions.

Alternative / Trade-off

We use full snapshots for small operational datasets and add change detection only when repeated full processing becomes materially expensive.

5. Lakehouse & Processing Platform

Once data crosses the ingestion boundary, all sources converge onto the same **S3 + Iceberg + Databricks/Spark** lakehouse.

5.1 Data Layers

Layer	Role	Typical Content
Raw	Immutable source evidence	Original files, API payloads, CDC/event history
Bronze	First managed source-aligned layer	Technical normalization + ingestion metadata
Silver	Trusted reusable layer	Deduplicated, validated, conformed entities
Gold	Consumer-oriented data products	Facts, dimensions, detailed business datasets, aggregates/KPIs

5.2 Platform Choices

Storage — Object Storage / Amazon S3

We use S3 as the durable storage foundation for Raw and Iceberg tables, with lifecycle policies moving older retained data into cheaper storage tiers.
Storage Format & Compression: Lakehouse tables use columnar Parquet with workload-appropriate compression such as ZSTD/Snappy. We balance storage and scan-I/O reduction against compression/decompression CPU cost, and combine compression with periodic small-file compaction.

Table Format — Apache Iceberg

We choose an **open table format** to add ACID transactions, schema/partition evolution, snapshots and multi-engine access on top of object storage.
We select **Iceberg** because interoperability across Spark, Flink and other engines is more valuable to this design than tighter platform coupling.
Delta Lake would be attractive for deeper Databricks-native integration, while **Hudi** is strong for CDC-heavy incremental workloads. We are not assuming Iceberg is universally faster; performance still depends on partitioning, file sizing, compaction and workload shape.

Batch / Micro-batch Compute — Databricks / Spark

We use managed distributed compute for CDC, files, API/spreadsheet processing, Bronze→Silver→Gold transformations and historical backfills.
Databricks Classic Compute gives us control over worker types, autoscaling and Spot/On-Demand strategy without operating Spark infrastructure ourselves.

We accept Databricks platform cost/coupling instead of building and operating an equivalent Spark/EMR platform.

Streaming — Flink

We reserve Flink for genuinely continuous **stateful/event-time** workloads; it is not the default engine for every source.

Orchestration — Airflow

Airflow coordinates schedules, dependencies, Spark jobs, retries, reconciliation and backfills. It orchestrates processing; it does not perform the heavy data transformation itself.

5.3 Build vs Buy

Capability	Decision
Spark/platform operations	Use Databricks
Object storage	Use S3
Table/storage semantics	Use Iceberg
Workflow orchestration	Use Airflow
Business transformations	Build
DQ contracts/rules	Build
Reconciliation logic	Build
Domain models/metrics	Build

Our principle is:

Buy commodity platform capability; build the business logic and correctness controls that differentiate the platform.

6. Data Reliability & Lifecycle

Section 4 handles reliability at the **source-ingestion boundary**. Here we define the guarantees that continue across Bronze, Silver, Gold and serving.

6.1 Idempotent Processing

We design processing units so the same input can be executed repeatedly without creating duplicate trusted data.

Depending on the workload, Spark writes use:

- deterministic partition replacement; or
- key-based **MERGE**/upsert into Iceberg.

Retries run the **same processing logic**; we do not maintain separate retry-specific write paths.

6.2 Retry & Partial Failure

We retry from a well-defined processing boundary rather than continuing from an arbitrary partially processed state.

Examples:

- file → replay the file;
- CDC → replay the landed change range;
- partition → recompute the partition;
- API payload → replay the immutable landed response.

Airflow manages retries and dependencies; Spark/Iceberg provide deterministic processing and atomic table commits.

6.3 Schema Evolution

We classify schema changes by impact:

Change	Handling
Optional field added	Accept where backward-compatible
Type widening / semantic change	Review impact and migrate
Required field removed / incompatible type / consumed field renamed	Block or quarantine

For migrations, we use:

add replacement → support old + new → migrate consumers → deprecate → remove

Schema Registry protects Kafka contracts; files/APIs/spreadsheets use contract validation; CDC uses schema comparison; Iceberg handles compatible table evolution downstream.

6.4 Reconciliation

We use reconciliation to detect unexpected loss, duplication or value corruption across processing boundaries.

Depending on dataset criticality, controls include:

- source vs Raw counts;
- Raw vs Bronze counts;
- business-key counts;
- **SUM(amount_paise)** and other financial control totals;
- trusted input vs trusted output;
- selected real-time vs historical results.

For large datasets, we compare partition/control totals first and perform exact row-level comparison only for mismatching scopes.

6.5 Backfill & Historical Correction

We separate two cases.

Transformation defect

Raw/Bronze → corrected code → affected Silver/Gold

Source correction

We land the corrected source data as a **new immutable version** and reprocess the affected period.

For Gold aggregates, we recompute the affected scope rather than manually patching values.

6.6 Streaming Recovery

Flink checkpoints preserve:

- Kafka source position;
- keyed/operator state;
- window state.

On failure:

restore latest successful checkpoint → restore state/source position → continue

Kafka consumer offsets alone are not our Flink recovery mechanism.

Checkpoints handle recovery; watermarks handle event-time progress and late data.

Very late or historically corrected events remain recoverable through immutable S3 history.

6.7 Regional Recovery

We choose **active-passive regional recovery** under the current minute-level recovery assumption rather than the additional complexity of active-active processing.

Recovery covers:

- S3/Iceberg data;
- catalog metadata;
- Airflow metadata/configuration;
- schema configuration;
- Flink checkpoints;
- infrastructure and secrets references.

Exact RTO/RPO remains a production-validation item because it depends on replication guarantees and business criticality.

6.8 Reliability Guarantees & Boundaries

Guarantee	How we enforce it	Boundary
Safe retry / no duplicate trusted writes	Stable input identity + idempotent MERGE / deterministic overwrite	Requires stable source/business keys
Replayability	Immutable S3 Raw + versioned inputs	Only successfully landed data can be replayed
Schema safety	Contracts + Schema Registry + validation gates	Compatible changes may pass; breaking changes block
Trusted publication	DQ PASS/WARN/BLOCK gate	Applies to configured rules; not every anomaly is detectable
Reconciliation	Counts, keys, control totals and targeted exact comparison	Depth depends on dataset criticality
Streaming recovery	Flink checkpoints + transactional/idempotent sink	Exactly-once depends on sink semantics
Audit traceability	Source IDs, run IDs, lineage and audit tables	External lineage may require explicit integration

We therefore avoid absolute claims such as **“all data is exactly-once”** or **“all data is always correct.”** Each guarantee is stated together with the conditions under which it holds.

7. Trust & Governance

We treat trust as a combination of **discoverability, access control, data quality, provenance and observability**.

7.1 Metadata & Discovery

We use **Unity Catalog** as the central technical catalog for:

- schemas, tables and columns;
- ownership;
- business/technical metadata;
- lineage for supported Databricks workloads;
- governed access to Iceberg/S3 data.

External systems may require connector/API-based lineage registration; we do not assume every external field-level relationship appears automatically.

7.2 Security & Access Control

We apply **group-based least privilege** through the identity provider and Unity Catalog.

Persona	Access
Data Engineers	Required Bronze/Silver/Gold write access
Analysts / Finance	Governed Gold/metric access
Data Scientists	Silver/Gold read access
Auditors	Metadata, lineage and audit evidence; read-only
Applications	Serving interfaces only
Pipeline identities	Minimum required source/target access

We separate human and service identities, avoid shared credentials, and apply masking/row-level controls for sensitive data where required.

PII Handling: Sensitive columns are classified/tagged in the catalog and protected through least-privilege access, masking/tokenization and column/row-level policies. PII is propagated only where required and is excluded from logs, alerts and DQ error payloads where possible. Retention/deletion requirements remain driven by applicable regulatory and business policy.

Encryption: Data at rest—including S3 Raw/Iceberg data, Flink checkpoints, audit data and serving-store storage—is encrypted using platform-managed/KMS-backed keys. Data in transit between sources, ingestion, processing and serving components uses TLS. Workloads access keys and credentials through managed identities/secrets rather than embedding them in code.

7.3 Data Quality

We use **risk-based DQ**, not the same blocking policy for every dataset.

Critical datasets such as payments/finance receive strong blocking rules; important datasets block only critical failures; low-risk datasets primarily use monitoring.

DQ Contract → Pre-flight Validation → PySpark Rules → PASS / WARN / BLOCK → Trusted Publish

A **BLOCK** result prevents trusted publication.

7.4 Auditability & Provenance

We combine platform-native lineage/execution evidence with structured audit tables such as:

- **audit.ingestion_run**
- **audit.pipeline_run**
- **audit.dq_result**
- **audit.reconciliation_result**

• **audit.backfill_recovery**

The objective is to reconstruct:

Source → Raw → Processing Run → Silver/Gold → DQ/Reconciliation → Serving

7.5 Observability

We monitor:

- **Operational:** jobs, runtime, retries, failures, infrastructure.
- **Data:** freshness, volume, schema drift, DQ and reconciliation.
- **Business:** transaction volumes, financial totals and important domain KPIs.

7.6 AI-Assisted Monitoring & Incident Triage

Deterministic metrics, thresholds, DQ rules and reconciliation remain the authoritative monitoring controls. An AI-assisted operations layer can consume alerts, logs, lineage and runtime metadata to correlate related failures, summarize incidents, identify likely blast radius and suggest probable root causes or investigation steps.

For example, if multiple downstream pipelines fail after the same upstream schema change, AI can group the alerts and identify the common dependency rather than presenting each failure independently.

AI is advisory only: it does not override **BLOCK** decisions, alter financial data, or autonomously mark a failed pipeline as healthy.

Alerts → Deterministic Monitoring → Alert/Event → AI-assisted Triage → Engineer – correlate failures – summarize incident – blast-radius analysis └ probable root cause

8. Data Model & Serving

We model Gold around **business entities and consumer access patterns**, using dimensional models where appropriate.

8.1 Core Business Model

Representative entities include:

- Customer
- Payment / Transaction
- Recharge
- Loan
- Repayment
- Insurance Policy
- Claim

Where deterministic mappings exist, we use a canonical customer identifier across business domains; we do not assume probabilistic identity resolution.

Money remains stored as **integer paise**.

8.2 Gold Analytical Model — Star Schema

For analytical reporting, we use **star-schema style models** with explicit fact grain and conformed dimensions.

Representative facts:

- **fact_payment**
- **fact_recharge**
- **fact_loan_repayment**
- **fact_claim**

Representative dimensions:

- **dim_customer**
- **dim_date**
- **dim_product**
- **dim_provider**
- **dim_location**

Example Fact — fact_payment

Grain

One row represents one canonical payment transaction identified by payment_id .	
Type	Representative Columns
Keys	payment_id , customer_sk , product_sk , provider_sk , location_sk , payment_date_sk
Measures	amount_paise , fee_paise , tax_paise
Attributes	payment_status , payment_method , currency
Timestamps	created_at , authorized_at , completed_at , updated_at
Audit	source_system , source_version , pipeline_run_id

payment_id is the transaction business key; dimension surrogate keys support historical joins.

A payment may progress through:

INITIATED → AUTHORIZED → SUCCESS / FAILED / REFUNDED

We apply newer versions through an idempotent **MERGE** keyed by **payment_id**.

If status-transition analysis is required, we retain it separately, for example:

fact_payment_status_event

rather than changing the grain of **fact_payment**.

Example Dimension — dim_customer

Grain

One row represents one version of a canonical customer for a defined validity period.	
Type	Representative Columns
Surrogate key	customer_sk
Business key	customer_id
Attributes	customer_segment , city , risk_category , customer_status
Validity	valid_from , valid_to , is_current
Audit	source_system , source_version , pipeline_run_id

Example:

customer_sk		customer_id		segment		city		valid_from		valid_to		is_current
101		C123		Silver		Delhi		2025-01-01		2026-03-14		false
245		C123		Gold		Bengaluru		2026-03-15		NULL		true

When a historically relevant attribute changes:

close previous version → insert new dimension version

A payment retains the **customer_sk** valid at the payment timestamp, preserving historical reporting.

We use **SCD Type 1** for attributes where historical changes do not need to be retained.

8.3 Serving by Persona

8.3.1 Analysts / Finance

Gold Iceberg → Databricks SQL → Unity Catalog Metric Views (Semantic Layer) → BI / SQL / Finance

We use the **semantic layer** to define governed metrics, dimensions and business definitions consistently across reporting.

It is primarily for **Analytics / Finance / BI**; applications are not expected to consume the semantic model at runtime.

This path supports dashboards, governed KPIs, ad-hoc analysis, historical reporting and financial reconciliation.

8.3.2 Applications

Standard trusted consumption:

Gold / Databricks SQL → Service/API → Application

For high-concurrency analytical access or fresher aggregates:

Gold → ClickHouse

or:

Kafka → Flink → ClickHouse

Requirement-driven alternatives:

- **OpenSearch** for full-text/fuzzy search;
- **DynamoDB/Aerospike/Redis** for hard millisecond key-value access.

8.3.3 Data Scientists

Silver/Gold Iceberg → Databricks/Spark

Used for feature engineering, training datasets, experimentation and historical analysis.

We do not introduce an online feature store without an actual online-inference requirement.

8.3.4 Auditors

Unity Catalog + Audit Tables + DQ Evidence + Reconciliation Evidence

Auditors should be able to trace:

business result → Gold/Silver → pipeline run → Raw/source

including related DQ, reconciliation and access evidence.

9. Cost & Scaling

We optimize cost by **processing less data, reducing expensive movement/shuffle, and right-sizing compute before adding capacity**.

9.1 Spark Performance & Cost

Area	Optimization
1. Measure	Execution plan, runtime profile, shuffle/spill/skew, baseline runtime + cost; re-measure after changes
2. Read Less	Partition pruning, predicate pushdown, column pruning, columnar storage, incremental processing
3. Shuffle & Joins	Filter before join, broadcast small dimensions, join strategy/order, partial aggregation, avoid unnecessary repartition, handle skew
4. Execution & Storage	AQE, sensible partition count, selective cache/persist, executor memory/spill control, small-file control, output file sizing
5. Cluster & Cost	Right-size workers/cores/memory, autoscaling/dynamic allocation, ephemeral jobs, Spot where interruption is acceptable

We optimize the **query/data plan before adding compute**.

9.2 Kafka / Flink / Serving

Component	Main Scaling / Cost Controls
Kafka	MB/sec, partition throughput, consumer parallelism, replication, retention
Flink	operator throughput, backpressure, state size, checkpoint duration, sink throughput
S3 / Iceberg	compression, compaction, lifecycle tiers, incremental reads
ClickHouse	copy only required serving datasets; size for ingestion + query concurrency

9.3 Scale Growth

Growth	Response
2x	Scale compute/parallelism; architecture remains unchanged
5x	Re-check Kafka partitions, Flink state/checkpoints, Spark shuffle/skew, Iceberg files/metadata, serving concurrency
10x	Revisit partitioning, state/checkpoint strategy, sink throughput, compaction, sharding, data movement and storage lifecycle

We do not assume that 10x traffic automatically requires replacing the architecture; we benchmark the pressure points first.

10. Deep Dive 1 — Streaming Correctness

Problem

How do we handle duplicate, out-of-order and late events, and recover from a Flink failure without losing or double-counting business results? Our target is:

Recover from failure without losing or double-counting committed business results.

Scenarios We Must Handle

Scenario	Expected Behaviour
Same event arrives twice	Count/process it once
Events arrive out of order	Compute using event time, not arrival order
Event arrives late but within allowed window	Include it in the correct result
Flink crashes after processing records	Restore state and continue without double-counting
Sink write succeeds but processing retries	Repeated write must not create duplicate business output
Event arrives beyond streaming lateness window	Recover through historical reconciliation/backfill

Solution

Solution: Kafka → Flink → Transactional / Idempotent Sink

Stable `event_id` + bounded deduplication handles duplicates. Event time + watermark/allowed lateness handles disorder. Flink checkpoints preserve Kafka position and processing state. Transactional or idempotent sink writes prevent duplicate committed output after replay.

Events outside the streaming correction window are corrected through the historical S3/backfill path.

Guarantee: With stable event identity, successful checkpoints and transactional/idempotent sink semantics, recovery does not create duplicate committed business results.

Trade-off: A larger deduplication/lateness window improves correction coverage but increases Flink state, checkpoint size and recovery time.

Test: Inject duplicates, reordered/late events and a Flink crash; recovered output must equal the uninterrupted baseline with no duplicate contribution.

11. Deep Dive 2 — Scalable Data Quality Gate

Problem

How do we prevent critical bad data from reaching the trusted layer without applying expensive blocking checks to every dataset?

Our requirement is:

Known critical quality failures must never reach the trusted layer, while non-critical failures should not unnecessarily block publication.

Scenarios We Must Handle

Scenario	Expected Behaviour
Required column disappears	Stop before expensive processing
Breaking schema arrives	Quarantine / BLOCK
Critical uniqueness/completeness rule fails	Process may complete, but trusted publish must not happen
Non-critical threshold fails	Publish with WARN
Multiple rules fail	Highest severity determines outcome

Solution

DQ Contract → Pre-check → PySpark Rules → PASS / WARN / BLOCK → Publish Gate

Dataset criticality determines enforcement. Cheap contract/schema checks run before expensive processing; Spark evaluates data/business rules. **BLOCK** prevents the trusted writer; **WARN** publishes while recording the failure.

Guarantee: A dataset with a configured **BLOCK** failure cannot be published to the trusted layer.

Trade-off: Critical financial controls use exact checks; lower-risk monitoring can use cheaper thresholds or approximate checks where appropriate.

Test: Verify that breaking schema and critical rule failures block publication, while WARN conditions still publish and record the warning.

12. Deep Dive 3 — Large-Scale Reconciliation

Problem

How do we detect lost, duplicated or financially corrupted records at hundreds of millions of rows/day without full row-by-row comparison every run?

Our requirement is:

Detect disagreement cheaply, then perform exact comparison only where something is wrong.

Scenarios We Must Handle

Scenario	Expected Behaviour
One transaction missing	Detect mismatch and locate scope
One transaction duplicated	Detect mismatch
amount_paise changed	Financial control fails
Rows reordered	Reconciliation still passes
Large partition differs slightly	Narrow comparison to affected subset

Solution

Control Totals → Deterministic Key Buckets → Mismatching Buckets → Exact Diff

First compare row/business-key counts, `SUM(amount_paise)` and required control totals. If a partition differs, hash stable business keys into deterministic buckets and compare controls per bucket. Only mismatching buckets receive exact record-level comparison.

Guarantee: For fields covered by the reconciliation contract, mismatches are detected and localized before targeted exact comparison.

Trade-off: More control fields improve detection but increase reconciliation compute; critical financial datasets therefore receive stronger controls.

Test: Remove, duplicate or modify a transaction and verify the affected bucket is detected; reordering identical records must still pass.

13. Limitations, Open Validation & AI Disclosure

13.1 Deliberately Out of Scope

We do not introduce:

- online feature store / real-time ML serving;
- probabilistic identity resolution;
- custom ML anomaly-detection platform;
- exhaustive enterprise data model;
- active-active regional processing.

13.2 Production Validation Required

Before production sizing we would validate:

- actual freshness SLAs;
- application access patterns;
- CDC volume/burstiness;
- event/file sizes;
- Spark skew/workload profiles;
- concurrency;
- late-event distribution;
- regulatory requirements;
- formal RTO/RPO.

13.3 Likely First Pressure Points

At substantially higher scale we expect pressure first around:

- **Spark shuffle / skew** — large shuffles, uneven partitions, long-running tasks.
- **Flink state / checkpoints** — growing state, slower checkpoints, longer recovery.
- **Sink / serving throughput** — downstream systems become the bottleneck even if processing scales.
- **Iceberg file / metadata growth** — small files, manifest growth, compaction overhead.
- **Data movement** — cross-region/AZ transfer and repeated lakehouse-to-serving copies.

13.4 AI Assistance

- I used AI as a **devil’s advocate** to challenge assumptions, architecture choices and trade-offs, and to research/compare technology options where needed.
- AI helped **structure, compress and format** the design document.
- I defined the **architecture, implementation strategy, guarantees, failure scenarios and test approach**.
- The implementation code was **generated with AI assistance based on my design**, then reviewed and validated by me for correctness and alignment with the intended guarantees.